DELIVERY OPERATIONS ANALYSIS

Last-Mile Bottleneck Analysis

Dhyan Santhosh Bellary

CSE Undergrad to Operations and Strategy

April 2026

github.com/dhyanbellary/delivery-operations-optimization

Project Website

BACKGROUND

The Problem

Food delivery in India is a logistics problem that often looks like a technology problem.

Platforms like Swiggy and Zomato handle millions of orders every day. Customer satisfaction depends heavily on one number, delivery time. Once it crosses 35 minutes, ratings begin to drop. Cross 45 minutes, and the customer is usually lost.

Most discussions around slow delivery tend to blame traffic, distance, or weather.

I wanted to understand what actually drives delivery time using data instead of assumptions. So I built a dataset modeled on real operational parameters and broke down every minute of the delivery journey to identify where time is actually being lost.

"Where does time actually get lost in a food delivery?"

METHODOLOGY

The Dataset

I constructed a dataset of 250 delivery orders from scratch, modeled on how dark kitchen platforms actually operate.

The dataset includes 250 orders and 29 variables. Every stage of the delivery journey tracked from the moment an order is placed to the moment it reaches the customer.

Key variables tracked across every order:

Variable	What It Captures
Prep Time	How long kitchen takes per order type
Rider Assignment Lag	Gap between food ready and rider assigned
Pickup Lag	Time from rider assigned to actual pickup
Transit Time	Pickup point to customer door
Traffic Level	Low / Medium / High per order
Delivery Failure	Whether order failed and why

From the raw data I calculated 13 additional metrics including total delivery time, delivery speed, cost ratios, peak hour flags, and distance buckets. The full analysis was done in Microsoft Excel using pivot tables, a KPI dashboard, and 8 charts.

FINDING 01

Transit Time Dominates the Delivery Journey

Out of an average 57-minute delivery, 32.3 minutes is pure transit time. That's 56% of the entire journey just riding from restaurant to customer.

Breaking down where the 57 minutes go:

Phase	Time (min)	Share
Kitchen prep time	17.9	31%
Prep wait gap	2.7	5%
Pickup lag	4.1	7%
Transit time	32.3	57%

Transit is eating more than half the delivery. Everything else, including kitchen, waiting, and pickup, splits the remaining 25 minutes. This is the baseline finding everything else builds on.

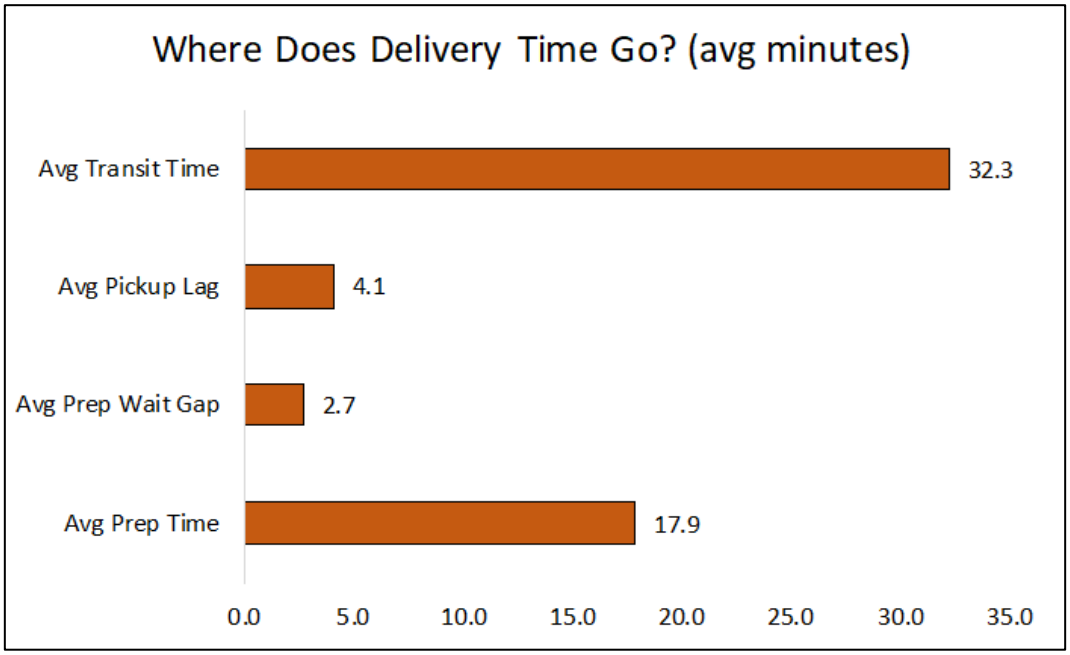

Figure 1: Average time spent in each delivery phase (minutes)

FINDING 02

Peak Hours Make Things 10% Worse

Orders placed during peak hours (lunch 12–2 PM and dinner 5–9 PM) take 59.6 minutes on average compared to 54.1 minutes during off-peak hours. This is a 5.5 minute difference, representing a 10.2% increase.

This isn't just about traffic getting heavier. Rider availability drops from 16.4 to 13.7 during peak windows while order volume spikes simultaneously. More orders chasing fewer riders. The math is simple and the impact is real.

Off-peak average: 54.1 minutes

Peak hour average: 59.6 minutes

Difference: 5.5 minutes (+10.2%)

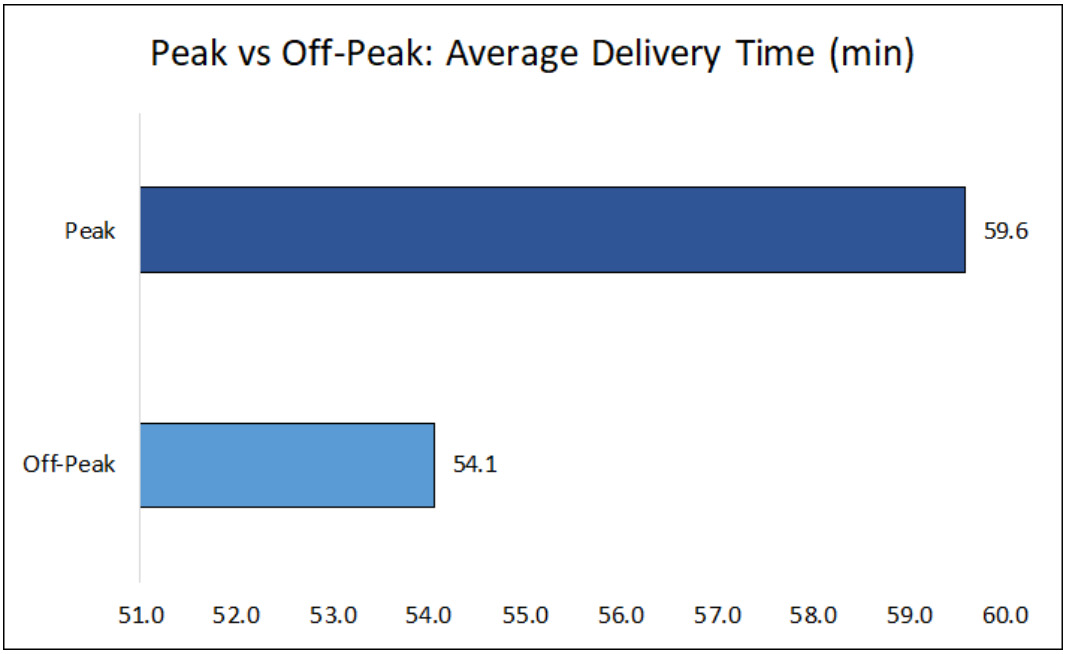

Figure 2: Average delivery time during peak vs off-peak hours

FINDING 03

Bulk Orders Have a Kitchen Problem, Not a Transit Problem

Bulk order prep time averages 32 minutes. Normal order prep time averages 15.3 minutes. That's a 16.7 minute difference which is more than double.

Here's what makes this finding important: transit time between bulk and normal orders is almost identical. 31.5 minutes vs 32.4 minutes. The road is not the problem.

The kitchen is overwhelmed by complex bulk orders and everything downstream gets delayed. Fixing transit time for bulk orders is solving the wrong problem.

Bulk order prep time: 32.0 minutes

Normal order prep time: 15.3 minutes

Bulk order total time: 75.4 minutes

Normal order total time: 53.6 minutes

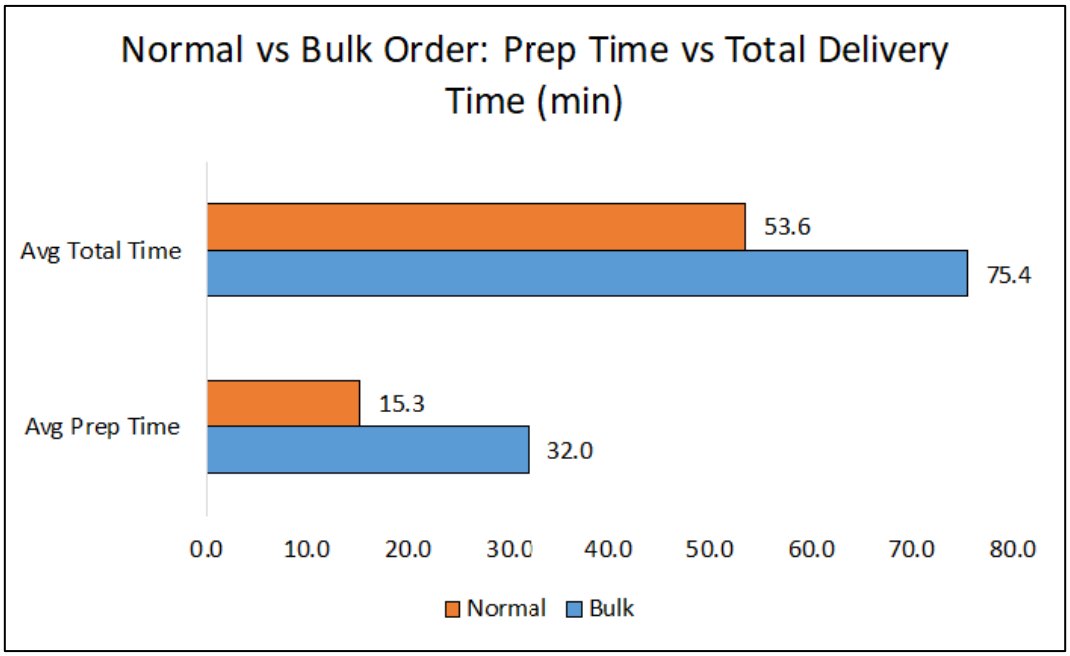

Figure 3: Prep time and total delivery time comparison by order type

FINDING 04

Distance Drives Transit Time More Than Traffic Does

High traffic orders average 31.5 minutes transit time. Medium traffic orders average 34.8 minutes. High traffic should be slower but it wasn't.

When I dug into the distance data, high traffic orders cluster at shorter distances (3.9km average vs 4.8km for medium traffic). The riders are stuck in worse traffic but covering less ground.

Distance is a stronger predictor of transit time than traffic level. Platforms that optimize purely for traffic routing are solving for the wrong variable.

High traffic average transit: 31.5 minutes (average distance 3.9km)

Medium traffic average transit: 34.8 minutes (average distance 4.8km)

Low traffic average transit: 27.0 minutes (average distance 4.3km)

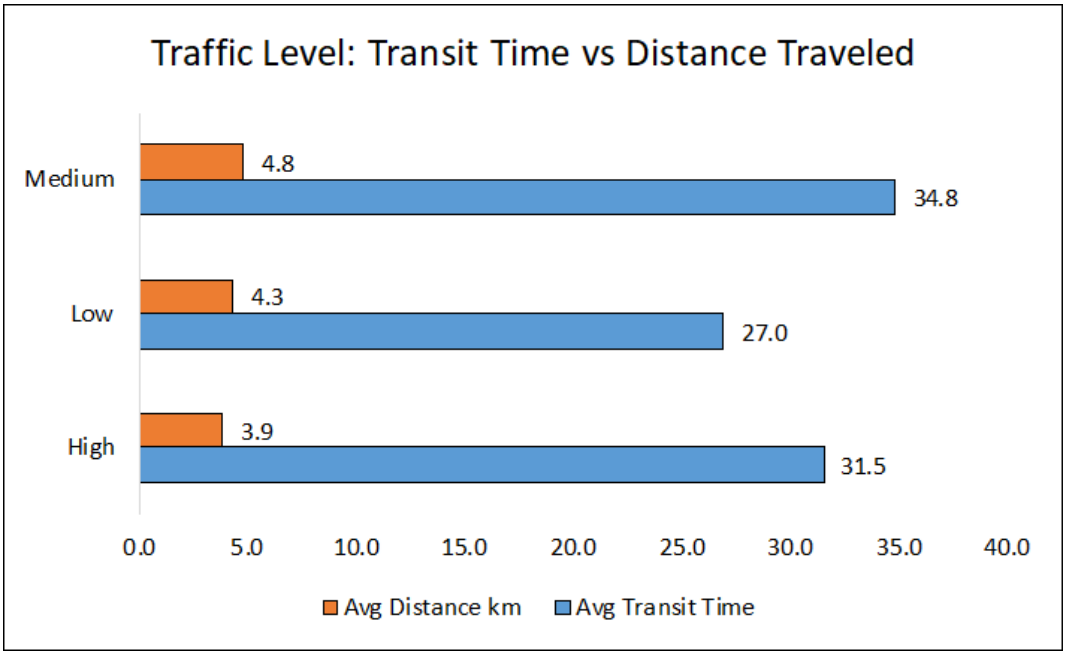

Figure 4: Transit time and distance traveled by traffic level

FINDING 05

Evening Is Where Failures Happen, Not Peak Night

The 4-6 PM window shows a 9.1% delivery failure rate, the highest of any time slot. Not peak night, but evening.

The kitchen gets overwhelmed before the dinner rush even starts. Orders fail before they leave the restaurant. By the time peak night arrives, the system has partially stabilized, which is why the peak night failure rate is only 3.4%.

The problem builds before it peaks operationally.

Late Morning failure rate: 7.7%

Afternoon failure rate: 6.5%

Evening failure rate: 9.1% (highest)

Peak Night failure rate: 3.4% (lowest)

All 17 failures in the dataset were caused by restaurant preparation delays, not traffic or distance. The kitchen is the single point of failure.

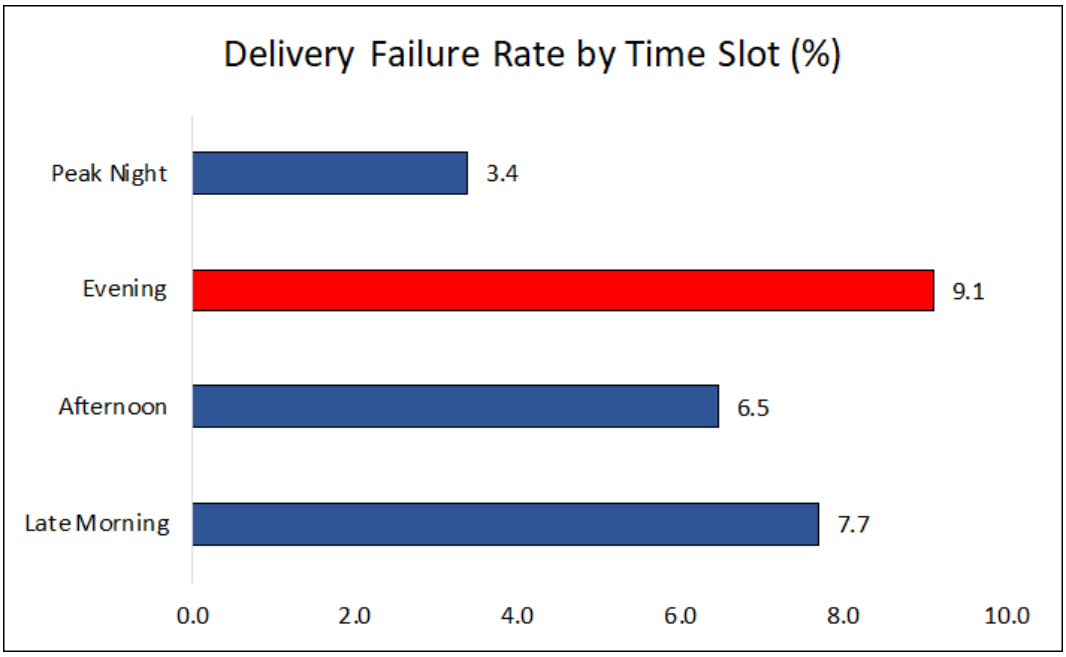

Figure 5: Delivery failure rate by time slot

FINDING 06

Rider Performance Varies by 15 Minutes Across the Same Platform

The best rider (R14) averages 22.6 minutes of transit time, while the worst rider (R11) averages 38.2 minutes. That's a 15.6 minute gap on the same platform, in the same city, handling the same types of orders.

This kind of variance doesn't happen randomly. Either routing is consistently allocating longer routes to specific riders, or there is a genuine performance issue that is not being monitored.

Either way, a 15-minute difference per order at scale represents an enormous amount of cumulative customer wait time that could be reduced without adding a single new rider.

Best rider (R14): 22.6 minutes average transit

Worst rider (R11): 38.2 minutes average transit

Gap: 15.6 minutes (38% difference)

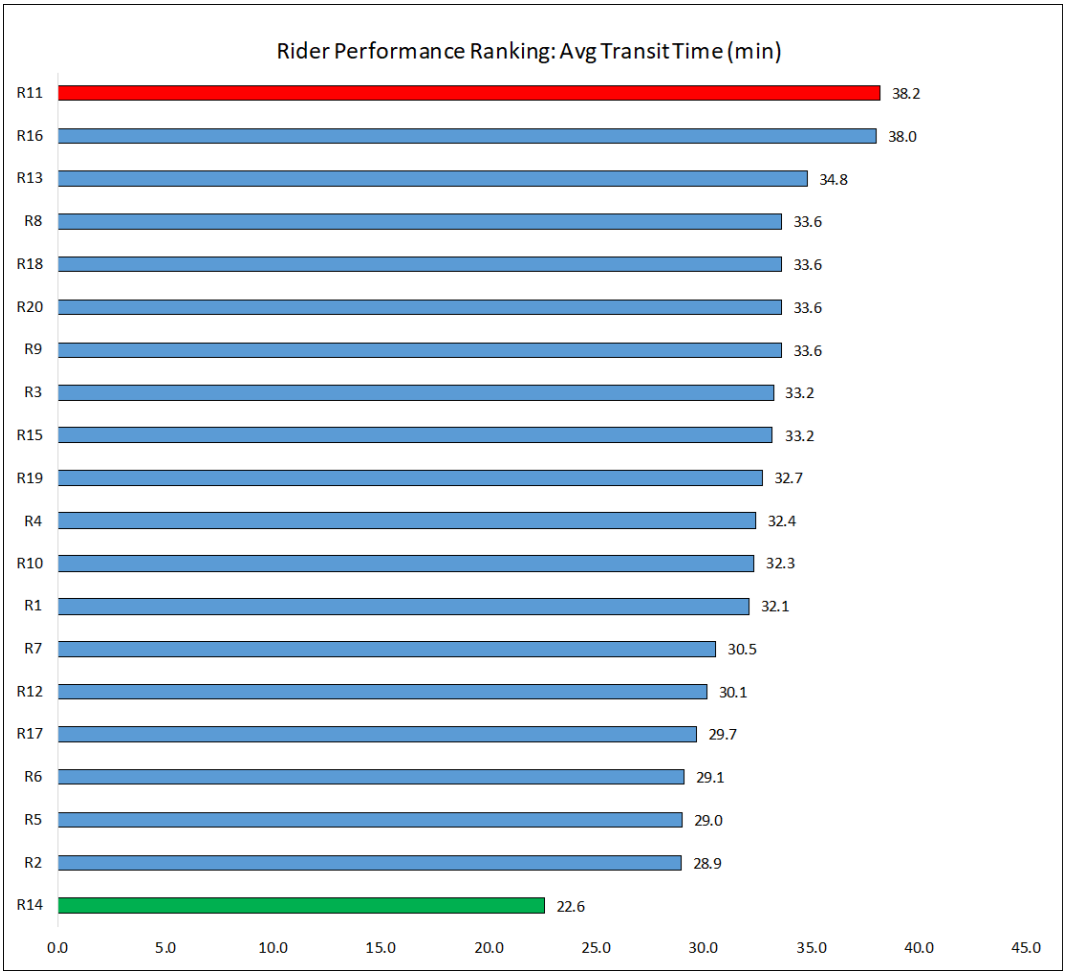

Figure 6: Rider performance ranking by average transit time

FINDING 07

7 PM Is the Operational Crisis Point

At hour 19 (7 PM), 35 orders hit the system simultaneously, the highest order volume of any hour. At the exact same time, average rider availability drops to 9.9, the lowest of any hour.

Peak demand and minimum supply collide at the same moment. Delivery time spikes to 63.5 minutes. The system has no buffer at this point. Any surge beyond expected volume would collapse delivery times entirely.

This is not just a busy period; it is a structural vulnerability.

Hour 19 order count: 35 orders (highest)

Hour 19 rider availability: 9.9 (lowest)

Hour 19 average delivery time: 63.5 minutes

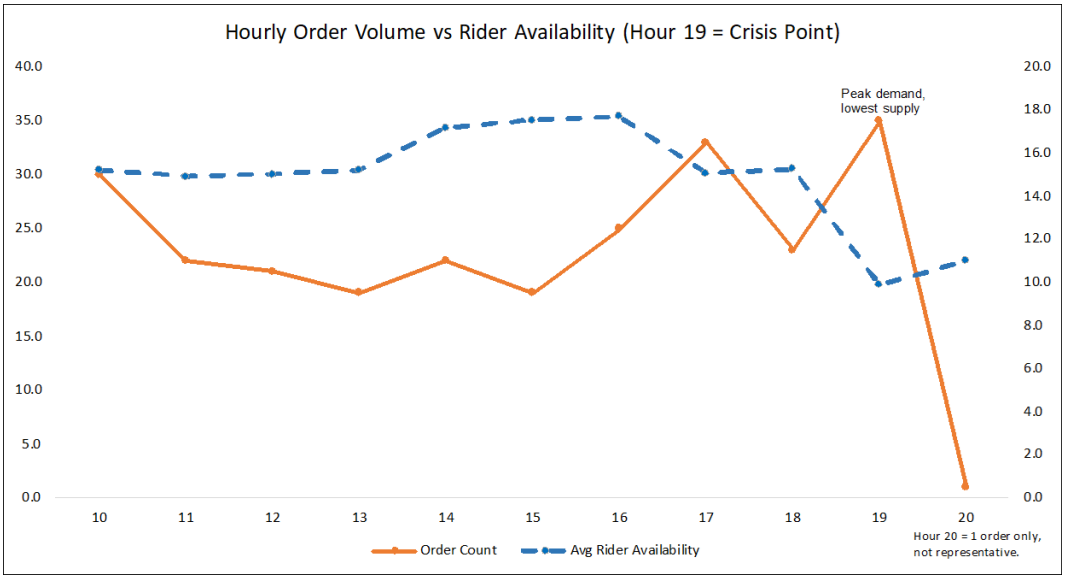

Figure 7: Hourly order volume vs rider availability (Hour 19 = crisis point)

FINDING 08

Short Distance Doesn't Mean Fast Delivery

Orders under 2km still take 43.4 minutes on average. Transit time for these short orders is only 17.5 minutes. The remaining 26 minutes is prep time and wait time for orders less than 2 kilometers away.

Distance was never the bottleneck. Kitchen operations and rider coordination are.

This finding reframes the entire optimization problem. Platforms that focus purely on routing and traffic optimization are addressing 17.5 minutes out of a 43-minute problem. The other 26 minutes lives in the kitchen and in coordination gaps.

Short distance average total time: 43.4 minutes

Short distance average transit time: 17.5 minutes

Non-transit time: 25.9 minutes (60%)

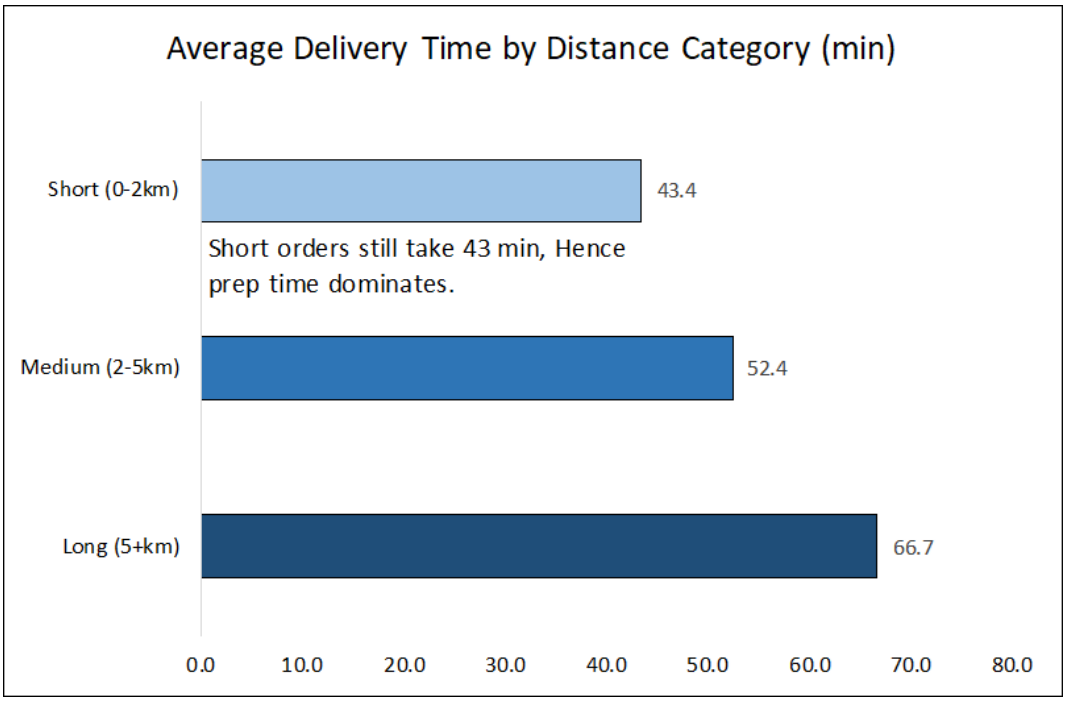

Figure 8: Average delivery time by distance category

What to Fix First

Five recommendations, in order of operational impact:

1. **Rider Surge at 5:30 PM, Not 6 PM, Not 7 PM**

   Action:

   Deploy 20% additional riders before hour 19 hits. Pre-positioning riders before demand peaks eliminates the supply-demand collision at 7 PM. Reacting after the peak starts is too late; the queue has already been built.

   Impact:

   -12% peak delivery time

2. **Kitchen SLA Alerts for Bulk Orders**

   Action:

   Set a prep time threshold per order type. Alert kitchen staff when bulk orders cross 25 minutes without completion. Riders arriving at restaurants to find food not ready create idle wait time that cascades into delays for subsequent orders.

   Impact:

   -30% bulk order failure rate

3. **Bottom-quartile rider monitoring**

   Action:

   R11, R16, and R13 consistently show the highest transit times. Investigate whether this reflects routing allocation or performance gaps. A 15-minute transit time difference at scale represents millions of minutes of cumulative customer wait time annually.

   Impact:

   -8 minutes average transit time

4. **Evening Pre-Assignment Strategy**

   Action:

   Assign riders to high-volume restaurants at 4 PM before the evening failure window opens at 5 PM. Proactive positioning prevents the kitchen overload failures that drive the 9.1% evening failure rate.

   Impact:

   Reduce evening failure rate from 9.1%

5. **Predictive Rider Assignment**

   Action:

   Do not wait for food to be ready before assigning a rider. Assign based on estimated prep completion time. This eliminates the 2.7-minute prep wait gap across all orders, small per order but significant at platform scale.

   Impact:

   -2.7 minutes per order across all orders

Limitations and Methodology Notes

What this analysis does not capture:

The dataset is self-constructed, not sourced from a live platform. Real operational data would include cuisine-type variance, GPS routing data, weather conditions, customer cancellation patterns, and restaurant-level performance metrics that this analysis lacks.

Hour 20 has only one order in the dataset. That data point is not statistically reliable, and the delivery time reading for that hour should not be interpreted as a meaningful pattern.

Rider performance variance might partially reflect routing decisions rather than pure skill or effort differences. Further analysis, controlling for distance allocation per rider, would be needed to fully separate the two effects.

The failure analysis covers 17 orders, a small sample. While the 100% restaurant-delay attribution is internally consistent, a larger dataset would be needed to confirm that this pattern holds at scale.

Tools Used

Microsoft Excel for data construction, cleaning, calculated columns, pivot table analysis, KPI dashboard, and chart creation.

GitHub for version control, project documentation, and hosting the public repository.

Website (portfolio): https://delivery-operations-optimization-by-dhyan.vercel.app

About

CSE undergrad transitioning into operations and strategy. I think in systems. I focus on identifying where processes break and how to fix them.

This is my first operations project. More to follow.

— Dhyan Santhosh Bellary